VENTURE LABS

Playbook, Rolle oder eigener Agent?

Der Leitfaden für alle, die gerade mit KI-Agenten anfangen: Was du wann brauchst, wie viel Trennung sinnvoll ist und wie du ohne Umwege startest.

Ohne Vorwissen | Mit Entscheidungshilfe | Mit Checkliste

ZUERST DAS WICHTIGSTE

Die Kurzfassung in 60 Sekunden

Wer anfängt, Agenten zu bauen, stolpert früh über dieselbe Frage: Reicht eine gute Anleitung, oder muss ich dem Agenten eine eigene kleine Welt bauen? Dieser Leitfaden beantwortet das.

BAUSTEIN 1

Playbook und Rolle

Derselbe Agent, dieselben Rechte, derselbe Ordner. Nur die Anleitung und die Fähigkeiten (Skills) ändern sich. Beispiel: «Teste das Backend» gegen «Teste das Frontend».

BAUSTEIN 2

Eigener Agent

Eigener Prozess oder eigener Container, eigener Ordner, eigene Schreibrechte, eigenes Modell, eigene Schlüssel, klar festgelegte Wege nach draußen.

BAUSTEIN 3

Quellen und Wissen

Dokumente, Wissensdatenbank, Listen. Sie sagen dem Agenten, was stimmt. Sie ändern weder Rechte noch Ablauf.

Die Faustregel

Fang mit dem Playbook an. Trenne erst, wenn es wehtut. Und es tut weh, sobald eine der drei Dinge zutrifft: andere Rechte nötig, ein Fehler kann teuer werden oder nach außen gehen, oder du brauchst ein anderes Modell.

Was drinsteht

Erst die Begriffe: Agent, Modell, Skill, Container	S. 3
Baustein 1: Playbook und Rolle	S. 4
Baustein 2: Der eigene Agent	S. 5
Baustein 3: Quellen und Wissen	S. 6
Was «getrennt» wirklich heißt: neun Schichten	S. 7 bis 9
Entscheidungshilfe: die Fragen in der richtigen Reihenfolge	S. 10
Der Spickzettel auf einer Seite	S. 11
Sechs Alltagsszenarien mit Empfehlung	S. 12 bis 13
Was wir selbst gelernt haben	S. 14
Fallstricke und was du lassen solltest	S. 15
So fängst du an: die Checkliste	S. 16
Quellen	S. 17
Wer wir sind und wie es weitergeht	S. 18

GRUNDLAGEN

Erst die Begriffe, damit wir dasselbe meinen

Jeder Fachbegriff hier bekommt einen Satz Erklärung. Wenn du einen Begriff schon kennst, überspring ihn.

Begriff	Einfach erklärt
Modell	Das «Gehirn» hinter dem Agenten, zum Beispiel Claude, GPT oder ein Modell, das auf deinem eigenen Rechner läuft. Es versteht Text und schreibt Text.
Agent	Ein Modell, das nicht nur antwortet, sondern **handelt**: Es liest Dateien, ruft Programme auf, schreibt Ergebnisse und entscheidet selbst, welcher Schritt als Nächstes kommt.
Anweisung (Prompt)	Der Text, der dem Agenten sagt, wer er ist und was er tun soll. Ein Playbook ist im Kern eine gut gepflegte Anweisung.
Skill	Ein Wissenspaket mit Anleitung, das der Agent bei Bedarf lädt, zum Beispiel «so schreibe ich ein Angebot». Ein Skill gibt dem Agenten Können, aber keine neuen Rechte.
Rechte	Was der Agent darf: welche Ordner er lesen, welche er beschreiben, welche Programme er starten, welche Konten er benutzen darf.
Prozess	Ein Programm, das auf dem Rechner läuft. Ein eigener Prozess hat eigenen Speicher und lässt sich einzeln starten, stoppen und überwachen.
Container (Docker)	Ein abgeschlossener Raum auf deinem Rechner mit eigenem Dateisystem. Der Agent sieht darin nur, was du ihm ausdrücklich hineinlegst oder einblendest. Wie ein Zimmer, in dem nur das steht, was du hineinstellst.
Schlüssel (API-Key, Token)	Ein Passwort für Programme. Wer den Schlüssel hat, darf den Dienst benutzen, und der Dienst rechnet darüber ab.
Wächter (Hook, Guard)	Ein kleines Programm, das vor einer Aktion prüft und sie bei Verstoß **verbietet**, statt den Agenten nur zu bitten, es zu lassen.
Freigabe (Approval Gate)	Ein Halt vor allem, was nach draußen geht. Der Agent bereitet vor, ein Mensch gibt frei.
Wissensdatenbank	Eine Sammlung geprüfter Dokumente, aus denen der Agent nachschlägt, statt aus dem Gedächtnis zu raten.

Wortverwirrung vorweg

Im Alltag heißen sehr verschiedene Dinge «Rolle» oder «Agent». In diesem Leitfaden gilt: **Eine Rolle ändert die Anweisung. Ein eigener Agent ändert die Welt, in der gearbeitet wird** (Ordner, Rechte, Modell, Schlüssel). An dieser Unterscheidung hängt alles Weitere.

BAUSTEIN 1

Playbook und Rolle: gleicher Agent, andere Anleitung

Ein Playbook ist ein Textdokument, das beschreibt, wie eine Art von Aufgabe erledigt wird. Eine Rolle ist ein Playbook mit Namen und Blickwinkel: «Du bist heute der Backend-Tester.»

Das Beispiel: Zwei Tester im selben Projekt

Backend und Frontend liegen im selben Projektordner. Beide Tester brauchen **dieselben Rechte**: Ordner lesen, Tests starten, Bericht schreiben. Sie bekommen nur **unterschiedliche Anleitungen und Skills**.

Rolle «Backend-Tester»

- Anleitung: Schnittstellen, Datenbank, Fehlerfälle
- Skill: API-Tests schreiben
- Ergebnis: Bericht mit Befunden

Rolle «Frontend-Tester»

- Anleitung: Klickwege, Anzeige, Barrierefreiheit
- Skill: Browser-Tests
- Ergebnis: Bericht mit Befunden

Dafür brauchst du keinen eigenen Container und keinen eigenen Prozess.
Sind Ordner, Rechte, Modell und Schlüssel gleich, wäre Trennung nur Mehraufwand.

Wann ein Playbook oder eine Rolle reicht

- Der Agent hat schon alles, was er braucht: Zugriff, Modell, Schlüssel.
- Es fehlt nur Know-how: ein Ablauf, ein Tonfall, eine Checkliste, ein Format.
- Ein Fehler ist billig und leicht rückgängig zu machen.

Was ein gutes Playbook enthält

1. **Ziel** in einem Satz: Was ist am Ende anders?
2. **Schritte** in der richtigen Reihenfolge, mit Entscheidungen («wenn X, dann Y»).
3. **Grenzen**: Was tut der Agent nie?
4. **Format und Beispiel**: Wie das Ergebnis aussieht, mit einem guten und einem schlechten Muster.

Die Grenze, die du kennen musst

Ein Playbook ist **eine Bitte, keine Sperre**. Steht dort «schreibe nie in den Kundenordner», hält sich der Agent in den meisten Fällen daran. Ob er es *kann*, entscheiden die Rechte. Was nie passieren darf, gehört deshalb nicht nur in den Text, sondern in eine echte Sperre (ab Seite 7).

BAUSTEIN 2

Der eigene Agent: eine eigene kleine Welt

Ein eigener Agent ist mehr als eine andere Anleitung. Er bekommt eine abgegrenzte Umgebung, in der nur das möglich ist, was du ihm gibst.

Was er bekommt

- **Eigenen Ordner** mit festgelegtem Lesen und Schreiben
- **Eigenen Prozess** oder **eigenen Container**
- **Eigenes Modell**, passend zu Aufgabe, Kosten und Datenschutz
- **Eigene Schlüssel** mit eigenem Budget
- **Definierte Wege** zu anderen Agenten und Diensten
- **Eigenes Gedächtnis**, falls nötig

Was du dafür zahlst

- Mehr Einrichtung: Rechte, Schlüssel, Überwachung
- Mehr Übergaben: Was der eine weiß, muss er dem anderen mitteilen
- Mehr Betrieb: Jeder Agent kann ausfallen, veralten oder Geld kosten
- Mehr Pflege: Regeln müssen in jeder Kopie ankommen

Prozess oder Container?

Beides sind Wege, einen Agenten abzutrennen. Der Unterschied liegt in der Stärke der Trennung.

	Eigener Prozess	Eigener Container
Was ist das?	Ein eigenes Programm auf deinem Rechner mit eigenem Arbeitsordner und eigenen Einstellungen.	Ein abgeschlossener Raum mit eigenem Dateisystem, nur ausgewählte Ordner sind eingeblendet.
Stärke	Gut für getrennte Aufgaben, getrennte Schlüssel und getrennte Arbeitsordner.	Stärker: Der Agent sieht den Rest des Rechners gar nicht und kann nur auf freigegebene Netzwege zugreifen.
Aufwand	Gering. Starten, stoppen und überwachen geht mit Bordmitteln.	Höher: Docker, Images, Ordnerfreigaben, Netzwerk.
Sinnvoll, wenn	Der Agent einen anderen Auftrag hat und einen eigenen Ordner braucht, die Aufgabe aber überschaubar bleibt.	Der Agent fremden Code ausführt, mit sensiblen Daten arbeitet oder dort Fehler nicht auf den Rest durchschlagen dürfen.

So machen wir es

In unserem eigenen Cluster laufen die Agenten als eigene Prozesse, jeder mit eigenem Arbeitsordner und eigenen Wächtern. Datenbanken, die Modell-Weiche und Überwachungsdienste laufen in Containern. Ein Container ist bei uns also eine Entscheidung für Dienste mit eigenem Datenbestand, nicht automatisch für jeden Agenten.

BAUSTEIN 3

Quellen und Wissen: woher der Agent weiß, was stimmt

Ein Agent weiß nichts über dein Unternehmen, bis du es ihm sagst. Und er rät gern, wenn er es nicht weiß. Quellen nehmen ihm das Raten ab.

Der Unterschied zum Playbook

PLAYBOOK

Wie etwas gemacht wird. Abläufe, Tonfall, Format. Ändert sich selten.

QUELLE

Was gilt. Preise, Produktdaten, Vertragsbedingungen, Ansprechpartner. Ändert sich laufend.

Was als Quelle taugt

- Eine **Wissensdatenbank** mit geprüften Seiten: Produkte, Prozesse, Entscheidungen, häufige Fragen.
- **Listen** wie Preislisten, Absender, Lieferanten, Kontaktdaten.
- **Vergangene Entscheidungen**: Warum haben wir etwas so gemacht?
- **Externe Quellen**, wenn der Agent sie lesen darf: Normen, Herstellerangaben, Gesetzestexte.

Vier Regeln für gute Quellen

1. **Eine Wahrheit pro Frage.** Steht der Preis an drei Stellen, wird einer davon veraltet sein.
2. **Mit Datum und Herkunft.** Der Agent soll sagen können, woher er etwas hat. Nur dann kannst du es prüfen.
3. **Lesen ja, schreiben nur kontrolliert.** Wenn Agenten Wissen ergänzen dürfen, dann mit Spur und Freigabe.
4. **Zuerst nachschlagen, dann antworten.** Das gehört ins Playbook: «Suche zuerst in der Wissensdatenbank. Steht dort nichts, sag das.»

Ein Satz, der viel Ärger spart

Schreib in jedes Playbook: *«Wenn du es nicht in den Quellen findest, rate nicht, sondern sag, dass es fehlt.»* Das ist die billigste Qualitätssicherung, die es gibt.

Quellen sind keine Rechte

Eine Quelle zu lesen heißt nicht, dass der Agent die Datei ändern darf. Plane Lesezugriff und Schreibzugriff getrennt (nächste Seiten).

ISOLATION

Was «getrennt» wirklich heißt: neun Schichten

Trennung ist kein Schalter, sondern ein Stapel. Du entscheidest pro Schicht, wie viel du brauchst. Je höher das Risiko, desto mehr Schichten.

1

Ordnerzugriff: lesen oder schreiben

Die wichtigste und billigste Trennung. Gib jedem Agenten nur die Ordner, die er braucht, und unterscheide strikt: **lesen** darf er viel, **schreiben** nur in einen eigenen Arbeitsbereich.

Beispiel: Der Recherche-Agent liest den Projektordner und schreibt nur in «Recherche-Ergebnisse». Der Original-Ordner bleibt unberührt.

2

Eigener Prozess

Der Agent läuft als eigenes Programm mit eigenem Arbeitsverzeichnis. Du kannst ihn einzeln stoppen, sehen, was er tut, und sein Verhalten getrennt festhalten.

Beispiel: Zwei Agenten arbeiten am selben Code, aber jeder in einer eigenen Kopie des Projekts, damit sie sich nicht gegenseitig überschreiben.

3

Eigener Container

Die stärkste lokale Trennung. Der Agent sieht nur, was du in den Container einblendest. Ordner kannst du dabei auch nur lesbar einblenden.

Beispiel: Ein Agent führt fremden oder neu geschriebenen Code aus. Passiert dabei etwas Unerwartetes, bleibt es im Container.

4

Eigene Schlüssel und Zugänge

Jeder Agent bekommt **seinen** Schlüssel, mit nur den Rechten, die er braucht. Kein gemeinsamer Admin-Schlüssel für alle. So ist klar, wer was getan hat, und ein gestohlener oder fehlerhafter Schlüssel richtet nur begrenzt Schaden an.

Beispiel: Der Mail-Agent darf lesen und Entwürfe anlegen. Zum Senden hat er keinen Schlüssel.

ISOLATION, FORTSETZUNG

Netz, Modell, Budget

5

Netzwerk und definierte Wege

Wohin darf der Agent sprechen? Zu welchen Diensten, zu welchen anderen Agenten? Je weniger Türen, desto weniger kann schiefgehen. Wenn Agenten miteinander arbeiten, dann über **einen festgelegten Weg** (eine Übergabe, ein Postfach), nicht durch gegenseitiges Hineingreifen in Dateien.

Beispiel: Der Recherche-Agent meldet Ergebnisse in ein Postfach. Dort holt sie der nächste Agent ab.

6

Modellwahl

Nicht jede Aufgabe braucht das stärkste und teuerste Modell. Und nicht jede Aufgabe darf zu einem Anbieter in der Cloud. Ein eigener Agent kann ein **anderes Modell** haben als der Rest:

- stark und teuer für Planung und Urteile, bei denen ein Fehler weh tut,
- schnell und günstig für Sortieren und Zusammenfassen,
- lokal auf deinem Rechner, wenn Daten das Haus nicht verlassen sollen.

Vorsicht: Dieselbe Anleitung verhält sich bei einem anderen Modell oft anders (siehe «Unsere Erfahrungen», Seite 14). Nach jedem Modellwechsel mit echten Testfällen prüfen.

7

Budget und Grenzen

Ein Agent in einer Schleife kann viel Geld und Zeit verbrennen. Setze deshalb pro Agent eine **Obergrenze**: Anzahl der Arbeitsschritte, Kosten pro Lauf, Laufzeit, Monatsbudget am Schlüssel. Wird die Grenze erreicht, hält er an und meldet sich.

Beispiel: Ein Testlauf darf höchstens eine bestimmte Zahl an Schritten machen. Danach stoppt er und liefert einen Zwischenstand.

Zwischenfazit

Die Schichten 1 bis 7 bestimmen, **was ein Agent kann**. Die zwei letzten bestimmen, **was er trotz Können nicht tun darf**, und genau dort entscheidet sich, ob du ruhig schlafen kannst.

ISOLATION, SCHLUSS

Wächter und Freigaben: Sperren statt Bitten

8

Wächter, die verbieten, statt zu fragen

Ein Wächter (Hook, Guard) wird vor einer Aktion aufgerufen. Er prüft sie und sagt **nein**, ohne dass der Agent darüber verhandeln kann. Das ist der Unterschied zwischen «bitte nicht» im Playbook und «geht nicht» im System.

Typische Wächter:

- Schreiben außerhalb des erlaubten Arbeitsordners verbieten,
- das Veröffentlichen oder Löschen in Produktivsystemen verbieten,
- verhindern, dass Passwörter und Schlüssel in Ausgaben auftauchen,
- Sammel-Befehle verbieten, die versehentlich fremde Dateien mitnehmen.

Unsere Erfahrung: Ein Wächter, der eindeutig verbietet, wirkt zuverlässiger als eine Frage an den Agenten, die er jedes Mal neu abwägen muss.

9

Freigabe vor allem, was nach draußen geht

Nach außen heißt: Kundenkontakt, Veröffentlichung, Zahlung, Änderungen an Live-Systemen. Hier gilt unsere härteste Regel: **Der Agent bereitet vor, ein Mensch gibt frei.** Das ist Infrastruktur, keine Frage des guten Willens des Agenten.

Beispiel: Der Agent legt einen Entwurf an. Gesendet wird er von dir, mit einem Klick, nachdem du ihn gelesen hast.

Zusammen sehen

Schicht	Playbook / Rolle	Eigener Agent
1 Ordner	Gleich wie der Haupt-Agent	Eigener Bereich, Lese- und Schreibrechte getrennt
2 Prozess	Derselbe	Eigener
3 Container	Nein	Optional, bei Risiko
4 Schlüssel	Gleich	Eigene, minimal
5 Netz und Wege	Gleich	Festgelegt
6 Modell	Gleich	Frei wählbar
7 Budget	Gemeinsam	Eigene Grenze
8 Wächter	Gemeinsam	Eigene, passend zur Aufgabe
9 Freigabe	Immer, wenn etwas nach außen geht, egal was du gewählt hast	

ENTSCHEIDUNGSHILFE

Brauche ich ein Playbook, eine Rolle oder einen eigenen Agenten?

Geh die Fragen von oben nach unten durch. Die erste Frage, die du mit «Ja» beantwortest, entscheidet.

1. Braucht die Aufgabe andere Rechte oder einen anderen Ordner als der Agent schon hat?
Zum Beispiel Schreibzugriff, ein Konto, einen Kundenordner.

→ **Ja: Eigener Agent**
Eigener Ordner, eigene Rechte.

Nein ↓

2. Kann ein Fehler teuer werden oder nach außen gehen?
Kunde, Zahlung, Veröffentlichung, Löschen, Live-System.

→ **Ja: Eigener Agent + Freigabe**
Eigene Schlüssel, Mensch gibt frei.

Nein ↓

3. Brauche ich ein anderes Modell?
Günstiger, stärker oder lokal wegen Datenschutz.

→ **Ja: Eigener Agent**
Modell pro Agent festlegen.

Nein ↓

4. Braucht die Arbeit ein eigenes Gedächtnis oder läuft lange und darf andere nicht stören?
Eigener Arbeitsbereich, eigene Historie, eigene Laufzeit.

→ **Ja: Eigener Agent**
Eigener Prozess.

Nein ↓

5. Wechselt nur der Blickwinkel oder Auftrag bei gleichen Rechten?
Backend testen gegen Frontend testen, Texter gegen Lektor.

→ **Ja: Rolle**
Playbook mit Name und Skills.

Nein ↓

6. Fehlt dem Agenten nur Wissen oder ein Ablauf?

→ **Ja: Playbook**
Plus Quelle, wenn Fakten fehlen.

Zwei Hinweise

- Jede Antwort mit «eigener Agent» kann auch zu einem Container führen. Das entscheidest du an der Frage: «Wie schlimm wäre es, wenn der Agent etwas Unerwartetes tut?»
- Playbook, Rolle und Quelle **kombinierst** du. Ein eigener Agent hat immer auch ein Playbook und meistens Quellen.

SPICKZETTEL

Alles auf einer Seite

	Playbook	Rolle	Eigener Agent	Quelle
Was ist es?	Anleitung für eine Aufgabenart	Playbook mit Name und Blickwinkel	Agent mit eigener Umgebung	Wissen, das gilt
Ordner und Rechte	Unverändert	Unverändert	Eigene, getrennt nach Lesen und Schreiben	Nur Lesen, falls nötig
Prozess / Container	Nein	Nein	Eigener Prozess, bei Risiko Container	Nein
Modell	Gleich	Gleich	Frei wählbar	Egal
Schlüssel	Gleich	Gleich	Eigene	Egal
Aufwand	Minuten	Minuten	Stunden bis Tage	Laufend pflegen
Risiko bei Fehler	Gering	Gering	Begrenzt durch Schichten	Falsche Antworten
Wann?	Es fehlt nur ein Ablauf	Nur der Blickwinkel wechselt	Rechte, Modell oder Risiko unterscheiden sich	Fakten ändern sich
Typischer Fehler	Als Sperre missverstanden	Als Rechtetrennung missverstanden	Zu früh, zu viele	Veraltet, doppelt

Die fünf Merksätze

1. **Text ist eine Bitte, eine Sperre ist ein System.** Was nie passieren darf, braucht eine Sperre.
2. **Gleiche Rechte, gleicher Ordner: Rolle.** Andere Rechte oder anderer Ordner: Agent.
3. **Jeder Agent bekommt seinen Schlüssel.** Kein gemeinsamer Admin-Zugang.
4. **Nach außen nie ohne Freigabe.** Der Agent bereitet vor, du gibst frei.
5. **Fang klein an.** Erst Playbook, dann trennen, wenn es wehtut.

SZENARIEN

Sechs Alltagsfälle mit Empfehlung (1/2)

Beispiele für kleine und mittlere Unternehmen. Die Empfehlung ist ein Startpunkt, kein Gesetz. Prüfe sie mit deinen eigenen Risiken.

1. Mails vorsortieren

Aufgabe	Der Posteingang wird nach Dringlichkeit und Thema geordnet, Entwürfe für Standardantworten entstehen.
Empfehlung	**Eigener Agent** mit Lesezugriff aufs Postfach, **Playbook** für Kategorien und Tonfall, **Quelle** mit Absenderlisten.
Rechte	Lesen und Entwürfe anlegen. Senden: nein.
Modell	Günstig reicht. Lokal, wenn die Mails vertraulich sind.
Sperre	Kein Sendeschlüssel. Versand nur durch dich.

2. Rechnungen und Belege

Aufgabe	Belege einlesen, zuordnen, auf Plausibilität prüfen, für die Buchhaltung vorbereiten.
Empfehlung	**Eigener Agent** mit eigenem Belegordner, im Zweifel im **Container**.
Rechte	Belegordner lesen, in «Zu prüfen» schreiben. Zahlungen: nein.
Modell	Lokal oder mit Vertrag zur Datenverarbeitung, weil Finanzdaten sensibel sind.
Sperre	Jede Zahlung und jeder Versand an Dritte nur nach deiner Freigabe.

3. Kundenantworten

Aufgabe	Anfragen beantworten, Standardfragen klären, Sonderfälle weiterreichen.
Empfehlung	**Playbook** (Tonfall, Ablauf) plus **Quelle** (Produkte, Preise, FAQ). Eigener Agent erst, wenn er selbst in den Kundenkanal schreibt.
Rechte	Quellen lesen, Entwurf schreiben.
Modell	Stark, weil Kunden den Text lesen.
Sperre	Geld, Verträge, Fristen und Rechtliches gehen immer an einen Menschen.

SZENARIEN

Sechs Alltagsfälle mit Empfehlung (2/2)

4. Änderungen an der Website

Aufgabe	Texte ändern, neue Seiten bauen, Fehler beheben.
Empfehlung	**Rollen** im selben Projekt (Texter, Prüfer) in einer **eigenen Arbeitskopie**, nicht im Live-Ordner.
Rechte	Projektordner, aber nur eine Arbeitskopie. Live-Zugang: nein.
Modell	Gleich für alle Rollen.
Sperre	Das Veröffentlichen übernimmt ein Mensch, nachdem er die Vorschau gesehen hat.

5. Code testen

Aufgabe	Backend, Frontend und Randfälle prüfen und Befunde melden.
Empfehlung	**Rollen** (Backend-Tester, Frontend-Tester), gleiche Rechte, gleicher Ordner, **kein Container nötig**.
Rechte	Projekt lesen, Tests starten, Bericht schreiben.
Ausnahme	Ändern die Tests Daten oder führen sie fremden Code aus, lohnt ein **Container** für den Testlauf.
Sperre	Obergrenze für Schritte und Kosten pro Lauf.

6. Recherche

Aufgabe	Markt, Wettbewerb, Fachthemen recherchieren und mit Quellen belegen.
Empfehlung	**Playbook** («jede Aussage mit Quelle, Unbelegtes kennzeichnen») plus **Quellenliste**. Eigener Agent nur, wenn Ergebnisse in ein gemeinsames Wissen fließen sollen.
Rechte	Web und Quellen lesen. Schreiben nur in den Recherche-Ordner.
Modell	Stark für die Auswertung, günstig für das Einsammeln.
Sperre	Ergebnisse tragen Quelle und Datum. Was nicht belegt ist, steht als «nicht belegt» da.

Das Muster hinter allen sechs

Je näher eine Aufgabe an **Geld, Kunden oder Veröffentlichung** liegt, desto mehr Trennung und desto mehr Freigabe. Je näher sie an **Denken und Vorbereiten** liegt, desto eher genügt ein Playbook.

AUS DER PRAXIS

Was wir selbst gelernt haben

Wir betreiben einen eigenen Agenten-Cluster für unsere Arbeit. Das hier sind Erfahrungen, keine Messreihen. Sie stammen aus unseren eigenen Entscheidungsprotokollen.

1. Aus Rollen wurden eigene Agenten

Wir haben anfangs mit Rollen gearbeitet (Planer, Umsetzer, Prüfer). Im September 2026 haben wir entschieden, jede Rolle zu einem **eigenen Agenten** zu machen: mit eigener Identität, eigenem Modell, eigenen Rechten und eigenem Budget. Begründung in einem Satz: Es sei die sauberere Einrichtung, wenn alles getrennt ist. Den Prüfer haben wir zuerst umgestellt, auf einen anderen Anbieter.

2. Lange Rollenketten kosteten mehr als ein Agent mit eigenem Ordner

Bei einer Aufgabe lief die Kette über viele Sitzungen und mehr als zwei Tage und scheiterte am Ende am Limit. Dieselbe Art Aufgabe erledigte ein einzelner Agent mit eigener Arbeitskopie in wenigen Minuten. Die Gründe: Jede neue Sitzung muss den Auftrag neu lesen, die Diagnose geht zwischen den Rollen verloren, und eine Rolle darf nichts außerhalb ihres Arbeitsbereichs ändern. **Das ist eine Erfahrung aus unserem Cluster, kein Vergleichstest.** Wir haben die Kette gekürzt, nicht die Rollen abgeschafft.

3. Dieselbe Anleitung, anderes Modell, anderes Ergebnis

Unser Prüfer lief einmal wegen eines Limits kurz auf einem anderen Modell, mit derselben Anleitung. Das neue Modell schrieb «Verdict: **approve**» mit Sternchen, unser Programm erwartete «Verdict: approve». Aus einer Freigabe wurde «Mensch nötig». Eine Zeile Formatunterschied. Lehre: **Playbooks sind nicht modellneutral.** Nach jedem Modellwechsel mit echten Fällen prüfen und Ausgabeformate tolerant einlesen.

4. Regeln erreichen nicht jede Kopie

Fünf unserer Prozesse rufen das Modell direkt auf und tragen ihre Anweisung im eigenen Quelltext. Unsere zentrale Regeldatei erreicht sie nicht. Wer eine Regel ändert, muss **jede Stelle finden**, an der sie gilt. Je mehr eigene Agenten, desto wichtiger wird dieses Verzeichnis.

5. Sperren schlagen Bitten

Unsere Agenten laufen in eigenen Arbeitskopien mit Wächtern, die zum Beispiel das Schreiben außerhalb des Arbeitsbereichs, das Ausgeben von Schlüsseln und das Veröffentlichen verbieten. Und: Nichts geht an Interessenten oder Kunden ohne Freigabe. Das haben wir als Infrastruktur gebaut, nicht als Bitte an den Agenten.

FALLSTRICKE

Typische Fehler und was du lassen solltest

Die häufigsten Fehler

- **Zu früh trennen.** Fünf Agenten für eine Aufgabe, die ein Playbook gelöst hätte.
- **Zu spät trennen.** Ein Agent mit allen Rechten für alles, bis etwas passiert.
- **Text für eine Sperre halten.** «Schreibe nie in den Kundenordner» steht im Playbook, aber der Ordner ist beschreibbar.
- **Rollen ohne Übergabe.** Jede Rolle fängt bei null an, die Erkenntnisse der letzten gehen verloren.
- **Modell gewechselt, nichts geprüft.** Das Format ändert sich, und niemand merkt es.
- **Quellen doppelt gepflegt.** Zwei Preislisten, zwei Wahrheiten.
- **Keine Obergrenze.** Eine Schleife läuft über Nacht und kostet Geld.

Was du lassen solltest

- **Keine gemeinsamen Admin-Schlüssel.** Jeder Agent bekommt seinen Schlüssel mit minimalen Rechten.
- **Kein Agent schreibt an Kunden ohne Freigabe.** Nie. Auch nicht «nur kurz».
- **Keine Passwörter oder Schlüssel in Anweisungen.** Was im Prompt steht, kann in Ausgaben und Protokollen landen. Schlüssel gehören in einen Tresor und werden zur Laufzeit geladen.
- **Keine Live-Änderungen ohne Mensch.** Veröffentlichen, Zahlen, Löschen: ein Mensch gibt frei.
- **Keine Rechte «auf Vorrat».** Gib nur, was die Aufgabe heute braucht.
- **Keine Daten «zur Sicherheit».** Was der Agent für die Aufgabe nicht braucht, bekommt er nicht zu sehen.

Datenschutz in einem Absatz

Sobald ein Agent personenbezogene Daten verarbeitet (Kundenlisten, Mails, Bewerbungen), gelten die üblichen Regeln: Rechtsgrundlage klären, nur nötige Daten geben, Anbieter mit Vertrag zur Auftragsverarbeitung wählen, oder lokal rechnen. Das ist ein Grund, **das Modell pro Agent zu wählen**. Es ersetzt keine Rechtsberatung.

LOS GEHT'S

So fängst du an: die Checkliste

In dieser Reihenfolge. Jeder Schritt dauert Minuten bis Stunden, nicht Wochen.

Schritt 1: Eine Aufgabe wählen

- [ ] Eine Aufgabe, die du selbst schon oft gemacht hast und genau beschreiben kannst.
- [ ] Ein Fehler wäre ärgerlich, aber nicht teuer.
- [ ] Das Ergebnis lässt sich in zwei Minuten prüfen.

Schritt 2: Das Playbook schreiben

- [ ] Ziel, Schritte, Grenzen, Ergebnisformat und ein Beispiel auf einer Seite.
- [ ] Der Satz: «Wenn du etwas nicht in den Quellen findest, sag das, statt zu raten.»

Schritt 3: Rechte festlegen

- [ ] Welche Ordner darf der Agent lesen, welche beschreiben? Schreibzugriff nur auf einen eigenen Arbeitsbereich.
- [ ] Eigener Schlüssel mit Budgetgrenze, nie dein Hauptkonto.
- [ ] Kein Zugang zu Versand, Zahlung oder Live-Systemen.

Schritt 4: Eine Sperre bauen

- [ ] Wähle die eine Sache, die nie passieren darf, und baue dafür eine echte Sperre. Das ist wichtiger als jedes weitere Playbook.

Schritt 5: Probelauf mit Aufsicht

- [ ] Zehn echte Fälle durchlaufen lassen, jedes Ergebnis selbst lesen.
- [ ] Notieren, wo der Agent danebenlag, und das Playbook nachschärfen.

Schritt 6: Erst jetzt über Trennung nachdenken

- [ ] Brauchst du andere Rechte, ein anderes Modell oder Schutz vor Fehlern? Dann geh die Entscheidungshilfe auf Seite 10 durch.
- [ ] Nach jedem Modellwechsel dieselben Testfälle erneut laufen lassen.

> **Wenn du nur eine Sache mitnimmst**
>
> Mach heute **ein** Playbook für **eine** Aufgabe und gib dem Agenten nur Lesezugriff. Alles andere folgt aus dem, was du dabei merkst.

NACHLESEN

Quellen

Was wir zitieren, und wofür. Recherchestand 4. November 2026. Die Aussagen über unseren eigenen Cluster stammen aus unseren internen Protokollen und sind hier als Erfahrung gekennzeichnet.

1. **Claude Code Dokumentation, Subagents.** Jeder Subagent startet mit eigenem, isoliertem Kontext. Werkzeuge, Rechte und Modell lassen sich pro Subagent festlegen. https://code.claude.com/docs/en/sub-agents
2. **Anthropic, «Building effective agents».** Unterscheidung von Workflows (festgelegte Abläufe) und Agenten (steuern ihren Ablauf selbst). Rat: die einfachste Lösung wählen und Komplexität erst bei Bedarf ergänzen. https://www.anthropic.com/research/building-effective-agents
3. **PromptBridge (arXiv 2512.01420).** Beschreibt «Model Drifting»: Bei einem Modellwechsel verschiebt sich die Leistung bei gleichem Prompt. https://arxiv.org/abs/2512.01420
4. **«Prompt Engineering a Prompt Engineer» (arXiv 2311.05661).** Automatisch optimierte Prompts sind modellspezifisch und übertragen sich nicht verlässlich. https://arxiv.org/abs/2311.05661
5. **arXiv 2509.23387.** Der Gewinn einer Prompt-Verbesserung ist auf dem Modell am größten, auf dem sie optimiert wurde, und auf anderen Modellen kleiner. https://arxiv.org/abs/2509.23387
6. **arXiv 2512.22130.** Gegenbild: Innerhalb einer Modellfamilie übertragen sich Prompts oft gut. Portabilität sollte man deshalb mit eigenen Testfällen messen. https://arxiv.org/abs/2512.22130
7. **Eigene Erfahrung, Venture Labs.** Entscheidungsprotokolle zu Rollen und Agenten (September und Oktober 2026), Auswertung des Durchsatzes unseres Entwicklungsprozesses (Oktober 2026), Änderungsprotokoll des Entwicklungs-Managers. Keine veröffentlichten Messreihen.

Was dieser Leitfaden nicht behauptet

Wir haben **keinen Vergleichstest** «Playbook gegen eigenen Agent» gemacht. Ob ein Skill Rechte verändern kann, beantworten die Quellen unterschiedlich. Wir gehen deshalb von der sicheren Annahme aus: Rechte setzt das System, nicht der Text. Alle Zahlen und Fälle stammen aus unserem eigenen Betrieb und sind als Erfahrung zu lesen.

VENTURE LABS

WER WIR SIND

Venture Labs

Wir bringen mittelständische Unternehmen ins KI-Zeitalter. Aus der Praxis, mit einem Cluster, den wir selbst täglich betreiben.

AI Pulse in einem Absatz

AI Pulse ist unser Angebot in drei Schritten. Der **Pulse Check** ist die Diagnose: Wo steht dein Unternehmen, welche Prozesse lohnen sich für KI, was ist der erste sinnvolle Schritt? Im **Pulse Build** setzen wir diesen Schritt in einem vierwöchigen Sprint um. Mit **Pulse Care** betreuen wir die Lösung danach laufend. Du beginnst mit dem Check, den Rest entscheidest du danach.

Pulse Check: Wo fängst du an?

Wir sortieren gemeinsam, welche deiner Aufgaben ein Playbook brauchen, welche einen eigenen Agenten und welche noch keine KI. Du bekommst eine klare Empfehlung statt eines Berichts voller Fachwörter.

Termin buchen oder Fragen stellen:
venturelabs.team/de/kontakt
contact@venturelabs.team

Der Leitfaden ist eine Orientierung und keine Rechts- oder Sicherheitsberatung. Alle Fälle sind allgemein gehalten und enthalten keine Kundendaten. Stand: November 2026. © Venture Labs GmbH.